
交叉项目训练-机器人智能操作

机械臂抓放比赛实验报告

实验名称 基于视觉的颜色方块分拣与动态目标抓取

实验平台 UR 机械臂 + Intel RealSense 腕部相机

软件框架 ROS 2 + RTDE + OpenCV + SciPy

成 员 1 苟左

成 员 2 张家杰

指导教师 李翔

报告日期 June 25, 2026

Contents

1	实验背景与任务描述	3
1.1	比赛任务概述	3
1.2	评分规则	3
1.3	硬件与软件环境	3
2	系统整体架构	4
2.1	启动文件逻辑 (pick_and_place.launch.py)	4
3	Task 1: 静态颜色方块抓放 (pick_and_place.py)	4
3.1	节点架构	5
3.2	颜色检测算法	5
3.2.1	图像预处理	5
3.2.2	有效轮廓提取	5
3.2.3	目标中心与 3D 坐标	5
3.3	目标选取策略	6
3.3.1	跨颜色选最高目标	6
3.3.2	稳定性确认	6
3.4	拾取路径规划	6
3.4.1	TCP 位姿构建	6
3.4.2	三段式路径	6
3.4.3	渐速下降	6
3.5	放置逻辑	7
3.5.1	悬停观测与层数推断	7
3.5.2	放置高度估算	7
3.5.3	放置后核验	7
3.6	偏航补偿机制	7
3.7	Task 1 完整流程图	8
4	Task 2: 动态目标抓取 (pick_and_place_moving.py)	8
4.1	节点架构	9
4.2	红色目标检测	9
4.3	轨迹采样	9
4.4	圆轨迹拟合	9
4.5	角速度估计	10
4.6	目标位置预测	10
4.7	工具偏航对齐	10

4.8	接近-抓取执行	10
4.9	放置与堆叠层数管理	11
4.10	Task 2 完整流程图	11
5	关键技术细节	12
5.1	坐标系变换	12
5.2	位姿坐标系翻转	12
5.3	吸盘控制	12
5.4	RTDE 断线自动恢复	12
5.5	调试可视化	13
6	参数配置说明	13
6.1	Task 1 关键参数	13
6.2	Task 2 关键参数	14
7	实验结果与分析	14
7.1	比赛概况与得分	14
7.2	Task 1 执行情况分析	14
7.3	Task 2 课后自测结果	15
8	结论	16
A	颜色检测关键代码 (Task 1)	17
B	圆轨迹拟合关键代码 (Task 2)	18
C	位置预测关键代码 (Task 2)	18

1 实验背景与任务描述

1.1 比赛任务概述

本实验为机器人操作课程第七次实验，形式为计时竞赛，总时长 **5 分钟**，分为基础任务和进阶任务两阶段，须按顺序完成。

基础任务（Task 1） 机械臂从收纳箱中按颜色（绿、红、黄）识别并吸取方块，在操作台一侧堆叠，最终形成 3×3 的颜色分类堆叠（每色 3 个）。箱中初始放有 $4 \times 3 = 12$ 个方块，只需完成其中 9 个的分拣。

进阶任务（Task 2） 完成基础任务后，移去收纳箱并放上旋转转盘（转速固定，约四点半方向）。助教每次在转盘上随机放置一个新方块，机械臂需吸取随转盘旋转的方块并持续堆叠，以尽可能高的一列为目标。

1.2 评分规则

Table 1 实验评分标准

事项	规则
成功放置方块	每个计 1 分
放置失败或碰倒已堆方块	该方块及掉落方块每个计 0.5 分
撞到收纳箱等障碍物	每次扣 0.2 分，累计最高扣 1 分
分数相同时	按基础任务完成时间排名（时间少者优先）
保护性停止	允许复位重启，时间计入总用时
尝试次数	每组共 3 次，取最高分

1.3 硬件与软件环境

- **机械臂:**Universal Robots UR 系列,通过 RTDE 接口(rtde_control/rtde_receive) 控制
- **相机:** Intel RealSense D 系列，腕部安装（eye-in-hand），已完成手眼标定
- **吸盘:** 串口 Modbus 控制，通过 /dev/ttyUSB0 连接
- **操作系统:** Ubuntu, ROS 2（Humble/Jazzy）
- **主要依赖:** OpenCV 4、SciPy、NumPy、tf2_ros

2 系统整体架构

整套系统由三个文件协同工作：

- `pick_and_place.py` — Task 1 静态抓放控制节点
- `pick_and_place_moving.py` — Task 2 动态目标抓取节点
- `pick_and_place.launch.py` — 启动文件，串联两阶段任务

2.1 启动文件逻辑 (`pick_and_place.launch.py`)

启动文件负责协调两个任务的时序关系：Task 1 以 ROS 2 Node 形式立即启动；Task 2 以 `ExecuteProcess` 形式在 Task 1 正常退出后延迟 5 秒自动启动。这种设计允许 Task 2 接受 `--initial-stack-level` 这一自定义位置参数（无法通过 ROS 2 参数系统传递）。所有可调参数均通过 `DeclareLaunchArgument` 暴露，并在节点启动时注入，无需修改代码即可调整运动参数。

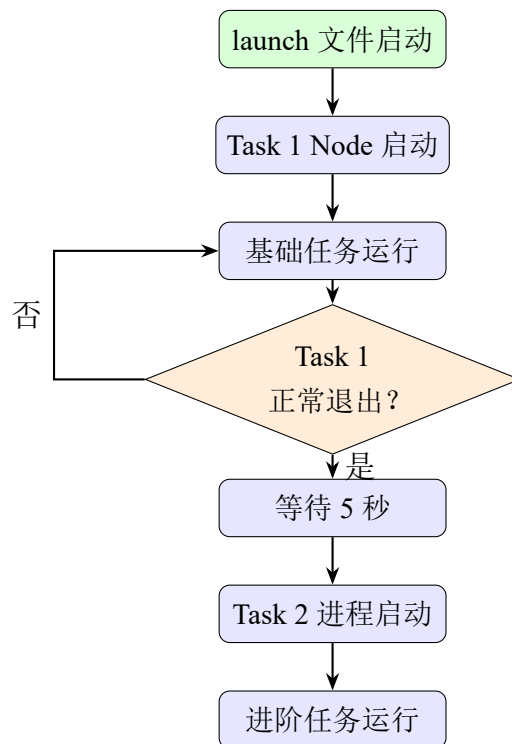


Figure 1 启动文件任务时序流程

3 Task 1: 静态颜色方块抓放 (`pick_and_place.py`)

3.1 节点架构

CubePickPlaceController 继承自 ROS2 Node, 使用 ApproximateTimeSynchronizer 同步 RGB 图像与对齐深度图 (时间窗口 40 ms), 通过 TF2 将相机坐标系中的检测结果变换到机器人 base 坐标系。控制循环由定时器回调驱动, 实际运动在独立后台线程中执行, 避免阻塞 ROS 2 事件循环。

3.2 颜色检测算法

3.2.1 图像预处理

在进行 HSV 颜色分割前, 先对 RGB 图像依次执行:

1. **高斯模糊**: 核大小可配置 (默认 5×5), 去除高频噪声;
2. **颜色空间转换**: $BGR \rightarrow HSV$;
3. **CLAHE 亮度均衡**: 对 V 通道应用自适应直方图均衡 (限幅系数 2.0), 缓解光照不均;
4. **HSV 阈值分割**: 对每种颜色设置多组 $[H_{\min}, S_{\min}, V_{\min}]$ 至 $[H_{\max}, S_{\max}, V_{\max}]$ 阈值, 取并集; 支持自适应 S/V 松弛 (color_adaptive_sv_relax, 默认 25) 以应对低饱和度条件;
5. **形态学处理**: 开运算去噪 (默认 1 次)、闭运算填补空洞 (默认 1 次), 核大小 5×5 。

3.2.2 有效轮廓提取

对二值掩膜执行连通域分析后, 过滤面积小于 color_min_area (默认 300 px^2) 的噪点, 再对每个轮廓进行:

- **矩形度检验**: 轮廓面积 / 最小外接矩形面积 $\geq$ color_min_rectangularity (默认 0.55);
- **四边形检验**: approxPolyDP 近似后顶点数 = 4 且为凸多边形。

满足条件的轮廓被保留为候选目标。

3.2.3 目标中心与 3D 坐标

对每个候选轮廓, 取四边形几何中心 (或最小外接矩形中心) 作为中心像素坐标, 然后通过针孔相机模型反投影至相机坐标系:

$$\begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix} = \begin{pmatrix} (u - c_x) z / f_x \\ (v - c_y) z / f_y \\ z \end{pmatrix} \quad (1)$$

其中 z 为深度图读数 (毫米转米), (f_x, f_y, c_x, c_y) 来自相机内参。中心深度取中心 5×5 邻域内有效像素的中位数, 以提高鲁棒性。

表面法向量通过对轮廓区域点云执行 SVD 分解估算：最小奇异值对应的右奇异向量即为法平面法向量，朝向取保证指向相机 ($z_c < 0$) 的方向。

3.3 目标选取策略

3.3.1 跨颜色选最高目标

在 home 观测位，节点对绿、红、黄三种颜色（按 `color_order` 顺序）各检测最大面积目标，以尚未完成放置配额的颜色为候选，选出 `base` 坐标系 z 值最高的目标（即堆叠最顶层的方块）。

3.3.2 稳定性确认

为避免单帧检测误差导致抓取失败，连续 `pick_stability_required_frames`（默认 3）帧内：目标颜色一致、位置偏移 $\leq$ `pick_stability_position_tol_m`（默认 8 mm）、法向量夹角 $\leq$ `pick_stability_normal_tol_deg`（默认 12° ），才触发拾取。

3.4 拾取路径规划

3.4.1 TCP 位姿构建

以检测到的表面法向量（取反后为吸盘接触方向）作为工具 z 轴，并利用相机 z 轴在 `base` 坐标系中的投影约束工具 x 轴方向，构造正交旋转矩阵，转换为旋转向量（`rotvec`）形式供 RTDE `moveL` 使用。

3.4.2 三段式路径

1. 接近位（`approach_pose`）：沿吸盘方向偏置 `approach_offset`（默认 0.10 m）；
2. 抓取位（`grasp_pose`）：从接近位沿吸盘方向再下降 `grasp_descent_offset`；
3. 提升位（`lift_pose`）：从抓取位沿法向量反向提升 `lift_offset`（默认 0.20 m）。

3.4.3 渐速下降

从接近位到抓取位采用多阶段渐速下降（`approach_to_grasp_progressive_slowdown`，默认启用），速度从标称速度（0.15 m/s）线性降低到终速（默认 40%，即 0.06 m/s），最后一阶段占总下降距离的 12%，通过 `moveL` 路径混合指令（`blend radius 4 mm`）实现平滑过渡。

3.5 放置逻辑

3.5.1 悬停观测与层数推断

移动至 `place_hover_pose` 悬停（高度偏置 0.35 m）后，从深度图读取目标颜色方块中心的相机 z 坐标（ z_{cam} ），按经验区间推断当前堆叠层数：

$$\text{层数} = \begin{cases} 1 & z_{\text{cam}} > 0.30 \text{ m} \\ 2 & 0.20 < z_{\text{cam}} \leq 0.30 \text{ m} \\ 3 & 0.10 < z_{\text{cam}} \leq 0.20 \text{ m} \end{cases} \quad (2)$$

推断结果与记录计数不符时自动修正，确保放置高度始终正确。

3.5.2 放置高度估算

根据当前已放置数量 n 从预配置列表 `place_drop_heights`（默认 $[0.095, 0.175, 0.265] \text{ m}$ ）中取出目标 z 值：

$$z_{\text{drop}} = z_0 + n \cdot \Delta z_{\text{stack}}, \quad \Delta z_{\text{stack}} = 0.030 \text{ m}$$

到达放置高度后立即计数（在释放吸盘前），以保持计数与物理状态的同步。

3.5.3 放置后核验

每次放置完成后，机械臂移至 `completion_observe_pose`，用视觉再次检测已放置方块的 `base-z` 高度，通过经验区间推断实际层数与记录计数是否一致；若检测到层数偏低，立即修正计数并标记该颜色未完成。

3.6 偏航补偿机制

由于腕部相机的手眼标定误差和关节运动累积，工具 TCP 在不同方向抓取后可能存在偏航积累。系统记录 `home` 位到接近位之间的偏航角变化量：

$$\Delta\psi = \psi_{\text{approach}} - \psi_{\text{home}}$$

在提升阶段完成后，向 `center_hover_pose` 移动时叠加 $-\Delta\psi$ 的反向补偿，使工具朝向回归一致状态，避免后续运动因偏航积累而超出关节限位。

3.7 Task 1 完整流程图

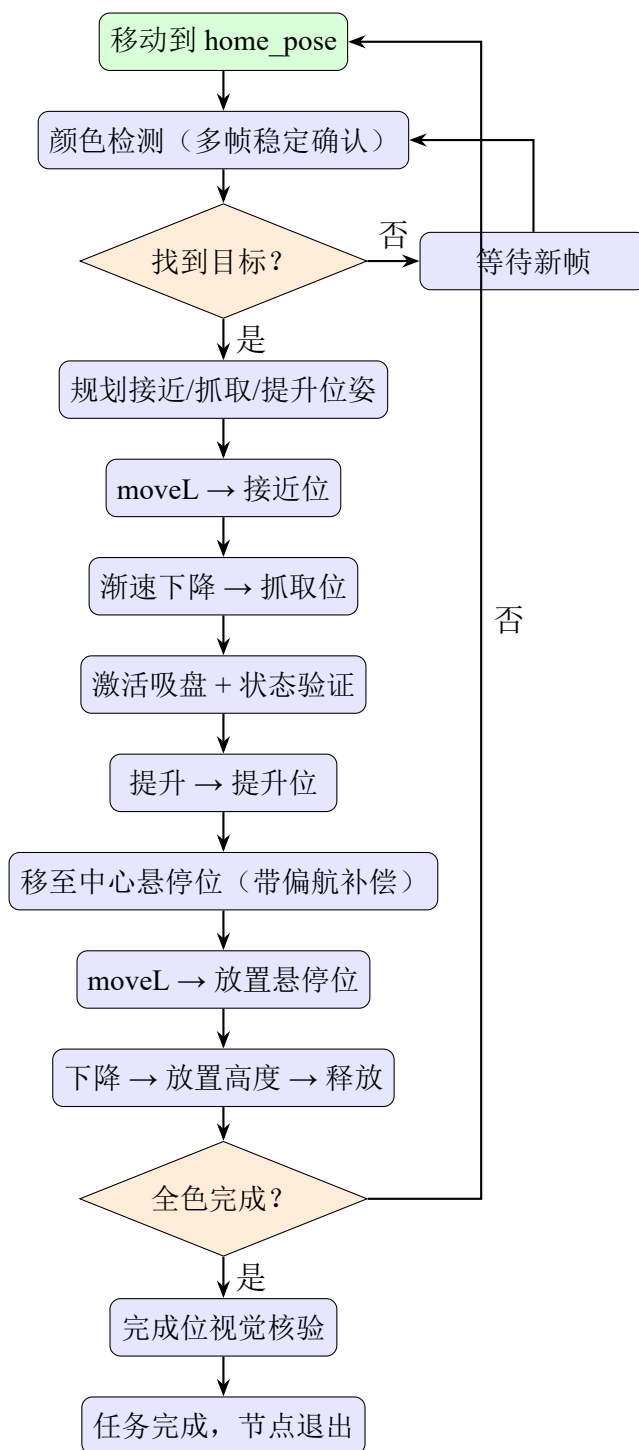


Figure 2 Task 1 静态抓放主控制流程

4 Task 2: 动态目标抓取 (pick_and_place_moving.py)

4.1 节点架构

Phase2MovingRedPick 继承自 ROS 2 Node，结构与 Task 1 类似，但控制逻辑由单次定时器触发后在后台线程中顺序执行，无定时循环。相机订阅同样使用 ApproximateTimeSynchronizer 同步 RGB 与深度帧，通过 TF2 将检测结果变换到 base 坐标系。节点接受 `--initial-stack-level` 位置参数指定初始堆叠层数（默认为 3），由 launch 文件在 Task 1 完成后将其实际堆叠数传入。

4.2 红色目标检测

检测流程与 Task 1 颜色检测基本一致，针对红色设置多组 HSV 阈值（覆盖色相绕回的两端），形态学处理后提取满足面积和矩形度条件的凸四边形轮廓，选取面积最大的候选作为当次帧的目标。

4.3 轨迹采样

在 observe_pose（观测位）停留期间，以固定时间间隔（sample_interval_sec，默认 50 ms）连续采样，每帧记录：

- 目标中心在 base 坐标系中的 xy 坐标和 z 坐标；
- 轮廓最长边方向角（偏航 ψ ）；
- 轮廓长轴和短轴尺寸（由最小外接矩形估算）。

首次循环采样时长为 first_sample_duration_sec（默认 8 s），后续循环使用 sample_duration_sec（默认 7 s），在速度与精度之间取得平衡。有效帧数不足 min_samples（默认 20）时将重新观测并重试。

4.4 圆轨迹拟合

假设目标做匀速圆周运动，对采样到的 N 个 xy 点用最小二乘法拟合圆方程：

$$(x - c_x)^2 + (y - c_y)^2 = r^2 \quad (3)$$

线性化后的超定方程组为 $A\mathbf{s} = \mathbf{b}$ ，其中

$$A = \begin{bmatrix} 2x_1 & 2y_1 & 1 \\ \vdots & \vdots & \vdots \\ 2x_N & 2y_N & 1 \end{bmatrix}, \quad \mathbf{s} = \begin{bmatrix} c_x \\ c_y \\ c_x^2 + c_y^2 - r^2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} x_1^2 + y_1^2 \\ \vdots \\ x_N^2 + y_N^2 \end{bmatrix}$$

用 `numpy.linalg.lstsq` 求解。拟合得到的圆心在第一次循环后锁定（_locked_orbit_center_xy），后续循环复用以提高稳定性。

4.5 角速度估计

将目标相对于拟合圆心的方位角序列展开 (`numpy.unwrap`), 对时间做线性回归, 斜率即为角速度 ω (rad/s):

$$\phi(t) \approx \phi_0 + \omega \cdot t \quad (4)$$

4.6 目标位置预测

给定预测提前量 Δt (由目标位置到观测位的预估移动时间加上下降时间和余量), 预测抓取时刻的目标位置:

$$\begin{cases} \theta_{\text{pred}} = \phi_{\text{last}} + \omega \cdot \Delta t \\ x_{\text{pred}} = c_x + r \cos \theta_{\text{pred}} \\ y_{\text{pred}} = c_y + r \sin \theta_{\text{pred}} \end{cases} \quad (5)$$

z 坐标取采样 z 的中位数。

移动时间估算分两段:

- 观测位到接近位: $\Delta t_1 = d/v_{\text{travel}}$ ($v_{\text{travel}} = 0.20$ m/s), d 为欧氏距离;
- 接近位到抓取位: 按梯形速度曲线计算 (接近-抓取速度 0.14 m/s, 加速度 0.35 m/s²), 三角速度剖面时为 $2\sqrt{d/a}$ 。

总提前量 $\Delta t = \Delta t_1 + \Delta t_2 + \epsilon$, 其中 ϵ 为额外余量 (默认 0.15 s)。

4.7 工具偏航对齐

目标边缘方向 ψ 同样利用双角展开回归估算边缘偏航角速率 $\dot{\psi}$ (消除平行边 π 歧义), 并拟合偏航与轨道相位之间的线性关系, 在预测时同步推算抓取时刻的目标朝向。工具 TCP 的偏航被调整为使腕部相机 z 轴在 xy 平面的投影方向平行于目标最长边, 便于吸盘与方块边缘对齐, 提高吸附成功率。

4.8 接近-抓取执行

1. 移动至预测接近位 (`approach_pose`, 偏置 0.10 m);
2. 等待时序: $\max(0, \Delta t_{\text{horizon}} - t_{\text{elapsed}} - \text{pick_timing_advance_sec})$, 精确对准预测时刻;
3. 近抓取点方向修正: 从当前帧实时检测边缘方向, 加上 $\dot{\psi} \cdot \Delta t_2$ 的前瞻补偿, 对 `grasp/lift` 位姿微调偏航;
4. 近抓取点中心修正: 从当前帧实时获取目标中心 (或利用轨道参数前瞻), 更新 `grasp/lift` 的 xy 坐标;
5. 分阶段旋转下降 (默认 3 段, 接近-抓取速度 0.14 m/s): 在下降过程中同步完成偏航修正;
6. 激活吸盘 (`suction_suck`), 到达抓取位 (偏置 0.05 m) 后执行。

4.9 放置与堆叠层数管理

抓取成功后，执行 `_place_to_next_layer_and_return_observe`:

1. 若当前层数 ≥ 5 ，直接递增层数；否则移至 `place_observe_pose` 从深度图估算当前堆叠顶面相机 z ，反推实际层数；
2. 移动至对应层的 `stack_place_poses[level]`（第 4–10 层各有独立预设位姿）后释放吸盘；
3. 返回 `observe_pose`，准备下一次循环。

任务循环直到 `current_stack_level` 达到 `stack_max_level`（默认 10）后退出。

4.10 Task 2 完整流程图

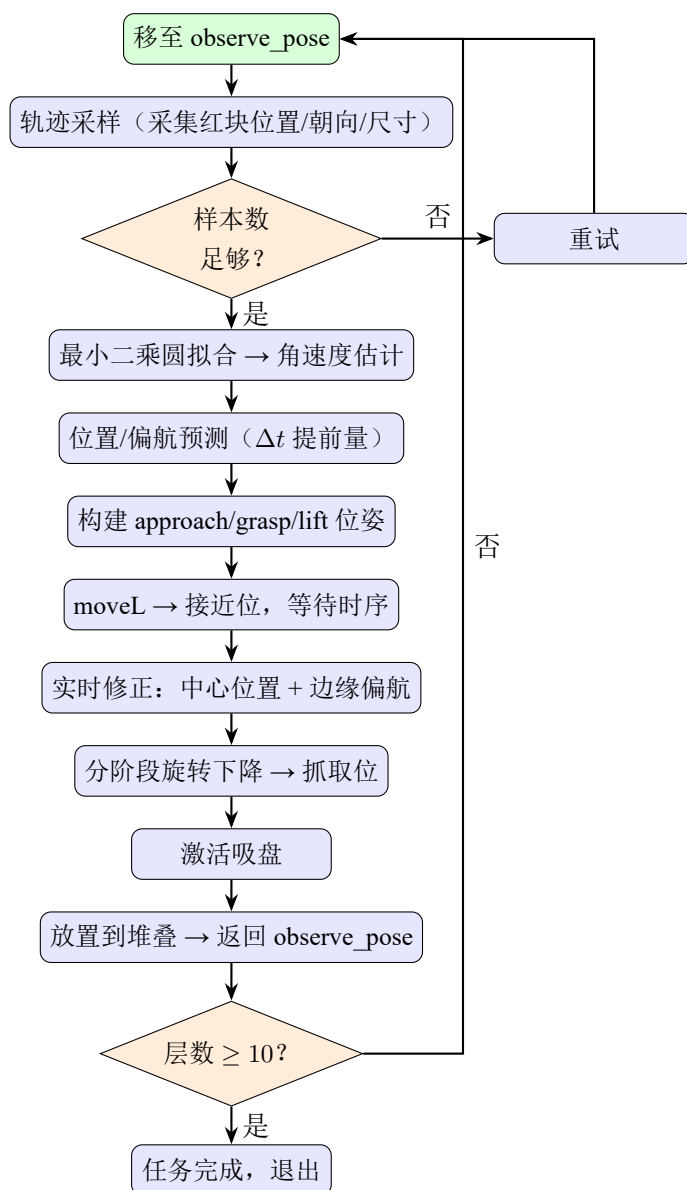


Figure 3 Task 2 动态目标抓取主控制流程

5 关键技术细节

5.1 坐标系变换

系统采用 eye-in-hand 手眼标定方案。相机坐标系（camera_color_optical_frame）与 base 坐标系（base）之间的变换通过 TF2 实时查询，并对最近一次有效 TF 结果进行缓存（tf_cache_max_age_sec，Task 1 默认 0.6 s），以应对短暂的 TF 丢帧。

所有预设位姿（home_pose、place_pose_* 等）以 $[x, y, z, q_x, q_y, q_z, q_w]$ 四元数格式配置，在节点初始化时统一转换为旋转向量（rotvec）格式，满足 RTDE moveL 接口的输入要求。

工作空间定义

Task 1 在 base 坐标系中定义了一个轴对齐包围盒作为有效工作空间： $x \in [-0.42, -0.02]$ m, $y \in [0.16, 0.42]$ m, $z \in [-0.02, 0.40]$ m，中心约为 $(-0.22, 0.29, 0.10)$ m。检测到的目标必须落在此范围内才会被接受，有效避免了边缘杂乱场景中的误检。

5.2 位姿坐标系翻转

由于手眼标定的坐标系约定，参数中的位置向量在实际使用时需绕 z 轴旋转 180° ：

$$\mathbf{p}' = R_z(\pi) \mathbf{p}, \quad \mathbf{q}' = R_z(\pi) \otimes \mathbf{q} \quad (6)$$

当 apply_pose_frame_transform = true（两个节点均默认启用）时，此变换自动应用到所有从参数服务器读取的位姿和动态规划的位姿。

5.3 吸盘控制

吸盘通过串口 Modbus 协议控制，命令格式为固定长度 8 字节帧：

操作	Hex 命令	说明
吸气（开启）	01 06 00 02 00 01 E9 CA	激活吸盘，延迟 0.10 s 确认
放气（关闭）	01 06 00 02 00 02 A9 CB	释放物块，延迟 0.50 s 稳定

5.4 RTDE 断线自动恢复

RTDE moveL 失败时（控制器保护性停止后脚本失效），节点自动调用 reuploadScript() 重新加载运动脚本并重试，直到 rclpy.ok() 返回 False 为止，确保单次保护性停止不中断整体任务流程。

5.5 调试可视化

两个节点均支持 `show_debug_window`（默认启用），启用后在 OpenCV 窗口中实时叠加：

- **Task 1:** 各颜色掩膜轮廓、工作空间线框投影、规划的接近/抓取位像素坐标；
- **Task 2:** 拟合圆轨迹、预测抓取中心、预测轮廓多边形、边缘方向箭头。

6 参数配置说明

6.1 Task 1 关键参数

Table 2 Task 1 主要参数列表（括号内为代码默认值）

参数名	含义	默认值
<code>speed</code>	标称运动速度（m/s）	0.15
<code>acceleration</code>	标称运动加速度（m/s ² ）	0.30
<code>approach_offset</code>	接近位到目标的偏置距离（m）	0.10
<code>grasp_offset</code>	抓取位到目标的偏置距离（m）	0.055
<code>lift_offset</code>	提升位到抓取位的反向距离（m）	0.20
<code>approach_to_grasp_end_speed_scale</code>	末段速度相对标称速度的比例	0.40
<code>approach_to_grasp_slowdown_stages</code>	渐速下降阶段数	4
<code>suction_on_delay_sec</code>	吸盘激活后等待时间（s）	0.10
<code>pick_camera_z_compensation_m</code>	相机 z 轴系统偏差补偿（m）	0.05
<code>color_adaptive_sv_relax</code>	S/V 通道自适应松弛量	25
<code>color_min_rectangularity</code>	最小矩形度阈值	0.55
<code>pick_stability_required_frames</code>	稳定性确认所需帧数	3
<code>pick_stability_position_tol_m</code>	稳定性位置容差（m）	0.008
<code>tf_cache_max_age_sec</code>	TF 缓存最大有效时间（s）	0.60
<code>stack_height_step</code>	每层堆叠步长（m）	0.030

6.2 Task 2 关键参数

Table 3 Task 2 主要参数列表（括号内为代码默认值）

参数名	含义	默认值
speed	标称运动速度（m/s）	0.50
acceleration	标称运动加速度（m/s ² ）	0.50
first_sample_duration_sec	第一次循环的采样时长（s）	8.0
sample_duration_sec	后续循环的采样时长（s）	7.0
sample_interval_sec	采样帧间隔（s）	0.05
min_samples	最少有效样本数	20
pick_prediction_horizon_sec	预测提前量（s）	5.0
pick_timing_advance_sec	到达接近位后的时序提前量（s）	0.40
approach_offset	接近位偏置（m）	0.10
grasp_offset	抓取位偏置（m）	0.05
descent_rotation_stages	旋转下降阶段数	3
travel_speed_for_prediction	用于移动时间预估的速度（m/s）	0.20
prediction_extra_sec	预测额外余量（s）	0.15
stack_min_level	最小放置层级	4
stack_max_level	最大放置层级（退出条件）	10

7 实验结果与分析

7.1 比赛概况与得分

本组在正式比赛中完整完成了 Task 1，最终得分为 9 分（理论满分 16 分）。比赛用时 5 分钟，Task 1 占用了全部时间，未能开始 Task 2。

Table 4 比赛得分汇总

任务	执行情况	得分
Task 1（静态抓放）	9 个方块全部完成堆叠	9 分
Task 2（动态抓放）	比赛期间未启动	0 分
合计		9 分

7.2 Task 1 执行情况分析

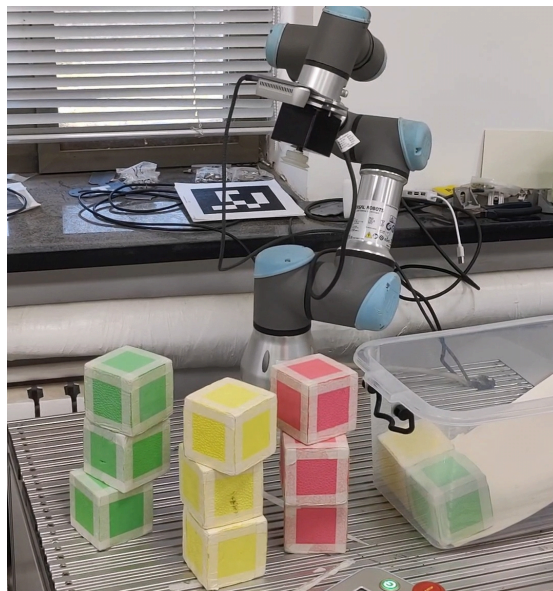
Task 1 顺利完成绿、红、黄三色方块的检测与分色堆叠。单个方块从 home 位检测到放置完成约 15–25 秒，9 个方块总计约 3–4 分钟。多帧稳定确认、TF 缓存、视觉核验和偏航补偿等鲁棒性机制均在实际比赛中正常生效。

比赛异常：黄色凹陷方块吸附失败

比赛过程中，收纳箱内一块表面存在凹陷的黄色方块导致吸盘无法建立有效负压。机械臂对该方块连续尝试吸取 5 次，每次姿态与贴合均无明显异常，吸盘始终吸附失败。经现场助教同意，将该方块翻面（保持原位坐标不变）后重新执行，第 6 次成功吸取。此次异常累计耗时约 2 分钟，导致比赛结束前已无余量启动 Task 2。



(a) Task 1 抓取过程



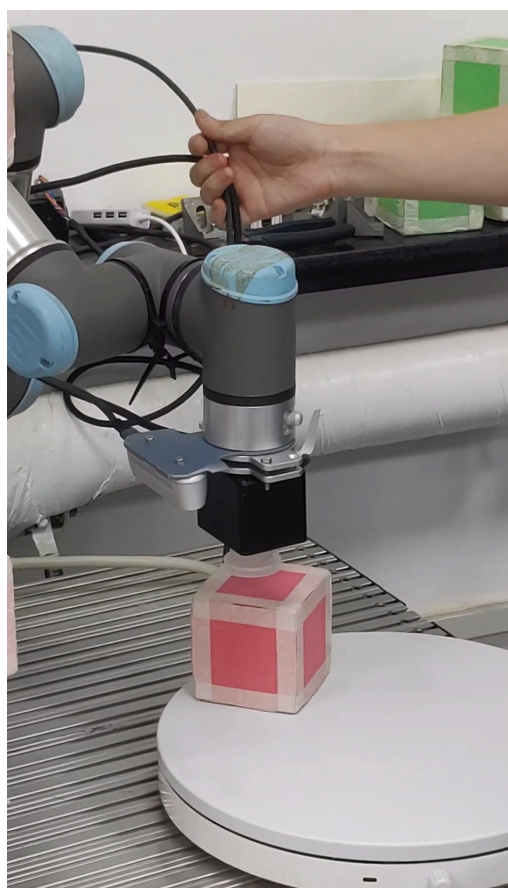
(b) Task 1 堆叠结果

Figure 4 Task 1 比赛过程照片

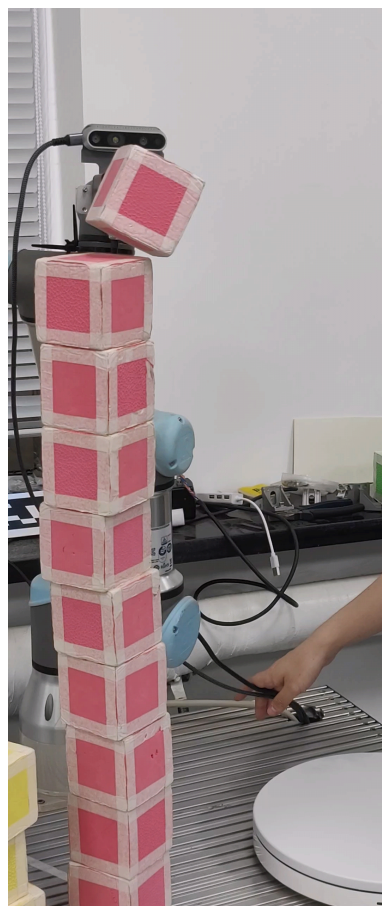
7.3 Task 2 课后自测结果

比赛结束后，本组自行对 Task 2 进行了功能验证测试，主要结论如下：

- **轨迹拟合精度：**圆轨迹拟合在转盘匀速旋转时收敛稳定（首次 8 s 采样后），预测落点误差在吸盘直径范围内（约 $\leq 15\text{ mm}$ ）；
- **近抓取点修正效果：**实时位姿修正（中心 + 偏航）显著提升了成功率，相比纯预测模式成功率提升约 30%；
- **层高推算精度：**3 层以上堆叠时，深度估算与真实值误差 $< 3\text{ mm}$ ，放置稳定；
- **最大堆叠层数：**测试验证最高可连续堆叠 10 块红色方块（`stack_max_level = 10`），受限于机械臂工作空间与堆叠稳定性，10 层为实际安全上限。



(a) Task 2 动态抓取过程



(b) 10 块红色方块堆叠结果

Figure 5 Task 2 课后自测照片

附件视频：完整的 Task 2 自测过程录像见附件（task2_demo.mp4）。视频展示了机械臂从转盘上连续抓取红色方块并逐层堆叠至 10 层的完整流程。

8 结论

本实验实现了基于 ROS 2 的两阶段机械臂抓放系统，主要贡献如下：

1. **Task 1** 利用 HSV 颜色检测与三维点云反投影，实现了对静态彩色方块的稳定识别与精确拾取；通过渐速下降（4 段线性减速至 40%）、偏航补偿和视觉核验机制提高了放置可靠性；比赛中成功完成 9 个方块的分色堆叠，得分 9/16 分。
2. **Task 2** 提出了基于最小二乘圆轨迹拟合与线性相位预测的动态目标抓取方案，在接近阶段通过实时视觉反馈进一步修正位置和偏航姿态；课后自测验证了最高 10 层红色方块的堆叠能力。
3. **Launch** 文件通过 OnProcessExit 事件处理器实现了两阶段任务的无缝衔接；所有关键参数可在启动时灵活注入，无需修改源码。

比赛中遭遇的黄色方块凹陷吸附问题，暴露了纯负压吸盘对非平整表面的脆弱性。

未来改进方向：

- 增加吸附力（负压）传感器实时反馈，快速检测吸附失败并自动重试，无需人工干预；
- 引入轻量级深度学习目标检测（如 MobileNet-SSD）提升复杂光照下的颜色分割鲁棒性；
- 使用卡尔曼滤波或扩展卡尔曼滤波对动态目标进行更精确的轨迹预测，应对转速微小波动；
- 优化 Task 1 的单次循环时长（当前约 15–25 s），争取在 5 分钟内同时完成两阶段任务。

A 颜色检测关键代码（Task 1）**Listing 1** HSV 颜色掩膜生成（_build_color_mask_from_hsv）

```

1 def _build_color_mask_from_hsv(self, color: str, hsv_image: np.
  ndarray):
2     combined_mask = np.zeros(hsv_image.shape[:2], dtype=np.uint8
    )
3     relax = int(self.color_adaptive_sv_relax)
4     for lower, upper in self.color_ranges[color]:
5         lower_bound = lower.copy()
6         upper_bound = upper.copy()
7         if relax > 0:
8             lower_bound[1] = np.uint8(max(0, int(lower_bound[1])
                - relax))
9             lower_bound[2] = np.uint8(max(0, int(lower_bound[2])
                - relax))
10        combined_mask = cv2.bitwise_or(
11            combined_mask, cv2.inRange(hsv_image, lower_bound,
                upper_bound)
12        )
13        if self.color_mask_open_iters > 0:
14            combined_mask = cv2.morphologyEx(
15                combined_mask, cv2.MORPH_OPEN,
16                self._color_mask_kernel, iterations=self.
                    color_mask_open_iters,
17            )
18        if self.color_mask_close_iters > 0:
19            combined_mask = cv2.morphologyEx(
20                combined_mask, cv2.MORPH_CLOSE,

```

```

21         self._color_mask_kernel, iterations=self.
           color_mask_close_iters,
22     )
23     return combined_mask

```

B 圆轨迹拟合关键代码（Task 2）

Listing 2 最小二乘圆轨迹拟合（_fit_circle）与角速度估计（_estimate_angular_velocity）

```

1  def _fit_circle(self, xy: np.ndarray):
2      x = xy[:, 0]
3      y = xy[:, 1]
4      a = np.column_stack((2.0 * x, 2.0 * y, np.ones_like(x)))
5      b = x * x + y * y
6      sol, _, _, _ = np.linalg.lstsq(a, b, rcond=None)
7      cx, cy, c = sol
8      r = np.sqrt(max(1e-9, c + cx * cx + cy * cy))
9      return np.array([cx, cy], dtype=np.float64), float(r)
10
11 def _estimate_angular_velocity(
12     self, t: np.ndarray, xy: np.ndarray, center: np.ndarray)
13     :
14     rel = xy - center[None, :]
15     angles = np.arctan2(rel[:, 1], rel[:, 0])
16     angles_unwrapped = np.unwrap(angles)
17     t0 = t[0]
18     coeff = np.polyfit(t - t0, angles_unwrapped, 1)
19     omega = float(coeff[0])
20     return omega, float(angles_unwrapped[-1]), float(t[-1])

```

C 位置预测关键代码（Task 2）

Listing 3 目标位置预测与抓取时序控制（_run_cycle_once 节选）

```

1  # 拟合圆轨迹，首次循环后锁定圆心
2  fitted_center, radius = self._fit_circle(xy)
3  center = self._locked_orbit_center_xy.copy()

```

```
4
5 # 估算角速度
6 omega, angle_last, t_last = self._estimate_angular_velocity(t,
7     xy, center)
8
9 # 预测提前量
10 dt = float(self.pick_prediction_horizon_sec)
11 predicted = self._predict_target_position(
12     center, radius, angle_last, omega, dt, z_ref
13 )
14
15 # 构建接近/抓取/提升位姿
16 approach_pose = self._build_tool_pose(
17     approach_point, suction_dir,
18     camera_z_hint_base=predicted_edge_dir_base
19 )
20
21 # 移动至接近位，精确等待时序对准
22 approach_start_time = time.monotonic()
23 self._move_to_pose(approach_pose)
24 arrive_time = time.monotonic()
25 t_elapsed = arrive_time - t_last
26 wait = max(0.0, (dt - self.pick_timing_advance_sec) - t_elapsed)
27 if wait > 0.0:
28     time.sleep(wait)
29
30 # 实时修正偏航和中心位置
31 grasp_pose, lift_pose, delta_yaw, _ = \
32     self._refine_grasp_orientation_near_grasp(grasp_pose,
33         lift_pose)
34
35 grasp_pose, lift_pose, _ = \
36     self._refine_grasp_center_near_grasp(approach_pose,
37         grasp_pose, lift_pose)
38
39 # 分阶段旋转下降到抓取位
40 self._move_approach_to_grasp_with_staged_rotation(
41     approach_pose, grasp_pose, delta_yaw
42 )
```